

#NUMPY

```
In [1]: import numpy as np
```

```
In [3]: np.__version__
```

```
Out[3]: '2.1.3'
```

```
In [4]: import sys  
sys.version
```

```
Out[4]: '3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:37:03) [MSC v.192  
9 64 bit (AMD64)]'
```

#Creating Numpy Array

```
In [5]: my_list = [0,1,2,3,4,5]  
my_list
```

```
Out[5]: [0, 1, 2, 3, 4, 5]
```

```
In [6]: type(my_list)
```

```
Out[6]: list
```

```
In [8]: np.array([3,2,75,18,8,0,35]) #1D array
```

```
Out[8]: array([ 3,  2, 75, 18,  8,  0, 35])
```

```
In [9]: a = np.array([3,2,75,18,8,0,35])  
print(a)
```

```
[ 3  2 75 18  8  0 35]
```

```
In [10]: type(a)
```

```
Out[10]: numpy.ndarray
```

```
In [11]: b = np.array([3,6,9,2,1,0,4])  
b
```

```
Out[11]: array([3, 6, 9, 2, 1, 0, 4])
```

```
In [12]: type(b)
```

```
Out[12]: numpy.ndarray
```

```
In [17]: a1 = np.array([ [23,89,45],[56,22,88]]) #2D Array  
print(a1)
```

```
[[23 89 45]  
 [56 22 88]]
```

```
In [15]: b1 = np.array([[1,2],[3,4]])  
b1
```

```
Out[15]: array([[1, 2],  
               [3, 4]])
```

```
In [16]: print(b1)
```

```
[[1 2]  
 [3 4]]
```

```
In [19]: a3 = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])    #3D -----#Te  
print(a3)
```

```
[[ 1  2  3  4]  
 [ 5  6  7  8]  
 [ 9 10 11 12]  
 [13 14 15 16]]
```

#dtype

```
In [21]: # The desired data-type for the array
```

```
In [22]: np.array([33,11,22] ,dtype =float)
```

```
Out[22]: array([33., 11., 22.])
```

```
In [23]: np.array([33,11,22] ,dtype = bool)    #Here True because ,python treats Non - z
```

```
Out[23]: array([ True,  True,  True])
```

```
In [24]: np.array([54,12,89.38] ,dtype = complex)
```

```
Out[24]: array([54. +0.j, 12. +0.j, 89.38+0.j])
```

```
In [26]: np.array([2.2,3.6,7.9,87.0] ,dtype = int)
```

```
Out[26]: array([ 2,  3,  7, 87])
```

#arange

```
In [27]: np.arange(1,21)    # 1-included,21-Last one got excluded
```

```
Out[27]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,  
               18, 19, 20])
```

```
In [29]: np.arange(0,10)
```

```
Out[29]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [66]: np.arange(-20,10)    #a < b  always
```

```
Out[66]: array([-20, -19, -18, -17, -16, -15, -14, -13, -12, -11, -10, -9, -8,
               -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5,
                6, 7, 8, 9])
```

```
In [67]: np.arange(20,10)
```

```
Out[67]: array([], dtype=int64)
```

```
In [34]: np.arange(1,30,2) #alternate numbers
```

```
Out[34]: array([ 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29])
```

```
In [35]: np.arange(0,40,3)
```

```
Out[35]: array([ 0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39])
```

```
In [36]: np.arange(0,21,2)
```

```
Out[36]: array([ 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
```

#Arange Reshape

```
In [38]: np.arange(1,11).reshape(5,2) #convert 5 rows and 3 columns and check also re
```

```
Out[38]: array([[ 1,  2],
               [ 3,  4],
               [ 5,  6],
               [ 7,  8],
               [ 9, 10]])
```

```
In [39]: np.arange(1,11).reshape(2,5)
```

```
Out[39]: array([[ 1,  2,  3,  4,  5],
               [ 6,  7,  8,  9, 10]])
```

```
In [41]: np.arange(1,13).reshape(3,4)
```

```
Out[41]: array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])
```

```
In [49]: np.arange(1,57).reshape(8,7)
```

```
Out[49]: array([[ 1,  2,  3,  4,  5,  6,  7],
               [ 8,  9, 10, 11, 12, 13, 14],
               [15, 16, 17, 18, 19, 20, 21],
               [22, 23, 24, 25, 26, 27, 28],
               [29, 30, 31, 32, 33, 34, 35],
               [36, 37, 38, 39, 40, 41, 42],
               [43, 44, 45, 46, 47, 48, 49],
               [50, 51, 52, 53, 54, 55, 56]])
```

```
In [75]: np.arange(1,13).reshape(12,1)
```

```
Out[75]: array([[ 1],
               [ 2],
               [ 3],
               [ 4],
               [ 5],
               [ 6],
               [ 7],
               [ 8],
               [ 9],
               [10],
               [11],
               [12]])
```

#ones & zeros

```
In [52]: np.zeros((3,5))    #we have to mention inside tuple
```

```
Out[52]: array([[0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0.]])
```

```
In [53]: np.zeros((5,3))    #parameter tuning
```

```
Out[53]: array([[0., 0., 0.],
               [0., 0., 0.],
               [0., 0., 0.],
               [0., 0., 0.],
               [0., 0., 0.]])
```

```
In [55]: a = np.zeros((3,5))
print(a)
```

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
```

```
In [68]: a = np.zeros((3,5) , dtype = int)    #hyperparameter tuning
print(a)
```

```
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
```

```
In [56]: np.ones((3,5))
```

```
Out[56]: array([[1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.]])
```

```
In [57]: b = np.ones((4,3) , dtype=int)
print(b)
```

```
[[1 1 1]
 [1 1 1]
 [1 1 1]
 [1 1 1]]
```

```
In [69]: print(type(a))
```

```
<class 'numpy.ndarray'>
```

```
In [70]: print(type(b))
```

```
<class 'numpy.ndarray'>
```

```
In [71]: np.zeros([2,9])
```

```
Out[71]: array([[0., 0., 0., 0., 0., 0., 0., 0., 0.],
               [0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

```
In [73]: np.zeros([2,9] ,dtype=int)
```

```
Out[73]: array([[0, 0, 0, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

```
In [72]: np.ones([4,7])
```

```
Out[72]: array([[1., 1., 1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1., 1., 1.]])
```

```
In [74]: np.ones([4,7] ,dtype =int)
```

```
Out[74]: array([[1, 1, 1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1, 1, 1]])
```

```
In [76]: n=[6,7]
          n1 = [6,8]                                # parameter tuning
          print(np.zeros(n1))
```

```
[[0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
```

```
In [78]: print(np.zeros(n,dtype = int)) #hyperparameter tuning
```

```
[[0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]]
```

```
In [79]: print(np.zeros(n))
```

```
[[0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0.]
```

```
In [80]: np.ones(3)
```

```
Out[80]: array([1., 1., 1.])
```

```
In [81]: np.ones(6)
```

```
Out[81]: array([1., 1., 1., 1., 1., 1.])
```

```
In [83]: np.ones(6,dtype = int)
```

```
Out[83]: array([1, 1, 1, 1, 1, 1])
```

```
In [84]: np.ones([5,6],dtype = int)
```

```
Out[84]: array([[1, 1, 1, 1, 1, 1],
                [1, 1, 1, 1, 1, 1],
                [1, 1, 1, 1, 1, 1],
                [1, 1, 1, 1, 1, 1],
                [1, 1, 1, 1, 1, 1]])
```

#Random

```
In [85]: np.random.rand(4)
```

```
Out[85]: array([0.618648 , 0.37579633, 0.7711753 , 0.02534323])
```

```
In [86]: np.random.random(3)
```

```
Out[86]: array([0.51087013, 0.66579596, 0.40033265])
```

```
In [87]: np.random.rand(8)
```

```
Out[87]: array([0.93204111, 0.79104083, 0.91650149, 0.99169805, 0.85435411,
                0.51709606, 0.90320169, 0.44556061])
```

```
In [88]: np.random.rand(2,4)
```

```
Out[88]: array([[0.007601 , 0.70751597, 0.76827219, 0.53213395],
                [0.9647906 , 0.50892792, 0.76509555, 0.0878732 ]])
```

```
In [89]: arr = np.random.randint(2,4)
```

```
In [90]: arr
```

```
Out[90]: 3
```

```
In [91]: np.random.rand(5,4)
```

```
Out[91]: array([[0.48838646, 0.41648611, 0.44740528, 0.3415535 ],
                [0.52651916, 0.36252464, 0.5007406 , 0.8865668 ],
                [0.31327417, 0.11985564, 0.36930693, 0.6215528 ],
                [0.57248106, 0.8265276 , 0.44979028, 0.88203574],
                [0.55705418, 0.30910078, 0.66949652, 0.0131102 ]])
```

```
In [93]: np.random.randint(4,5)  #(a,b) in this a<b always
```

```
Out[93]: 4
```

```

In [95]: np.random.randint(0,10)    # it gives any random no. 0-9

Out[95]: 7

In [96]: np.random.randint(10,20,5)

Out[96]: array([14, 14, 17, 19, 10], dtype=int32)

In [97]: np.random.randint(5,30,7)    # it gives 7 nos.between 5 to 30

Out[97]: array([28, 14, 24, 26, 25, 7, 13], dtype=int32)

In [100... np.random.randint(5)    #it gives any random no. before n-1

Out[100... 4

In [101... np.random.randint(-30,20,10)

Out[101... array([ 16, 4, -20, -4, -23, -1, 13, 8, -13, -23], dtype=int32)

In [102... a = np.random.randint(5,30,7)
print(a)

[ 5 19 9 11 7 5 13]

In [107... np.random.randint(10,40,(10,10))    #generate 10 to 30 with 10*10 matrix

Out[107... array([[20, 35, 15, 21, 28, 22, 39, 22, 20, 34],
      [38, 16, 12, 25, 27, 24, 26, 31, 29, 26],
      [39, 28, 35, 30, 34, 22, 34, 21, 29, 16],
      [27, 30, 32, 22, 10, 25, 28, 38, 16, 14],
      [14, 32, 38, 24, 17, 15, 19, 19, 20, 33],
      [30, 25, 11, 25, 19, 39, 30, 16, 20, 16],
      [32, 39, 31, 11, 17, 10, 11, 31, 30, 16],
      [16, 23, 16, 15, 17, 28, 26, 39, 33, 22],
      [20, 30, 18, 18, 37, 23, 18, 31, 36, 15],
      [12, 13, 25, 36, 23, 32, 32, 37, 17, 24]]), dtype=int32)

In [106... np.random.randint(2,9,(11,13))

Out[106... array([[8, 4, 7, 8, 6, 6, 4, 2, 6, 8, 6, 4, 6],
      [5, 4, 7, 3, 6, 8, 3, 3, 3, 3, 8, 4, 3],
      [4, 8, 8, 3, 7, 3, 7, 7, 2, 5, 3, 5, 4],
      [5, 6, 8, 2, 4, 8, 3, 2, 3, 8, 3, 7, 3],
      [4, 5, 7, 4, 3, 3, 2, 2, 2, 3, 7, 2, 8],
      [3, 8, 3, 6, 3, 7, 3, 3, 2, 3, 4, 3, 8],
      [3, 7, 4, 5, 2, 2, 3, 2, 4, 8, 4, 8, 2],
      [5, 3, 7, 5, 8, 8, 7, 7, 8, 8, 3, 7, 2],
      [4, 7, 7, 8, 7, 7, 3, 7, 3, 3, 4, 7, 3],
      [8, 6, 2, 5, 2, 8, 5, 3, 6, 4, 3, 5, 4],
      [6, 2, 5, 5, 7, 6, 3, 8, 2, 6, 6, 2, 5]]), dtype=int32)

```

#Random Reshape

```

In [108... np.arange(1,14).reshape(13,1)    #generate 13*1 matrix element between 1 to 14

```

```
Out[108... array([[ 1],
        [ 2],
        [ 3],
        [ 4],
        [ 5],
        [ 6],
        [ 7],
        [ 8],
        [ 9],
        [10],
        [11],
        [12],
        [13]])
```

```
In [109... b = np.random.randint(10,20,(5,4))
b
```

```
Out[109... array([[16, 10, 17, 12],
        [17, 18, 17, 16],
        [16, 13, 15, 15],
        [16, 12, 10, 17],
        [15, 10, 19, 10]], dtype=int32)
```

```
In [110... type(b)
```

```
Out[110... numpy.ndarray
```

```
In [121... np.arange(1,16).reshape(15,1)
```

```
Out[121... array([[ 1],
        [ 2],
        [ 3],
        [ 4],
        [ 5],
        [ 6],
        [ 7],
        [ 8],
        [ 9],
        [10],
        [11],
        [12],
        [13],
        [14],
        [15]])
```

```
In [122... np.arange(1,13).reshape(3,4)
```

```
Out[122... array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

```
In [123... np.arange(1,11).reshape(5,2)
```

```
Out[123... array([[ 1,  2],
        [ 3,  4],
        [ 5,  6],
        [ 7,  8],
        [ 9, 10]])
```

#linspace

```
In [125... # It is also called as linear space,given range at equal distance it creates poi
```

```
In [126... np.linspace(-10,10,10)    # Lower range,upper range ,no.of items to generate
```

```
Out[126... array([-10.          , -7.77777778, -5.55555556, -3.33333333,  
        -1.11111111,  1.11111111,  3.33333333,  5.55555556,  
         7.77777778, 10.          ])
```

```
In [127... np.linspace(-2,12,6)
```

```
Out[127... array([-2. ,  0.8,  3.6,  6.4,  9.2, 12. ])
```

```
In [128... np.linspace(2,20,8)
```

```
Out[128... array([ 2.          ,  4.57142857,  7.14285714,  9.71428571, 12.28571429,  
        14.85714286, 17.42857143, 20.          ])
```

```
In [132... a = np.linspace(1,20,8)  
print(a)
```

```
[ 1.          3.71428571  6.42857143  9.14285714 11.85714286 14.57142857  
 17.28571429 20.          ]
```

```
In [133... type(a)
```

```
Out[133... numpy.ndarray
```

```
In [134... np.linspace(1,10,5)
```

```
Out[134... array([ 1. ,  3.25,  5.5 ,  7.75, 10. ])
```

#Identity matrix

```
In [135... np.identity(3)
```

```
Out[135... array([[1., 0., 0.],  
        [0., 1., 0.],  
        [0., 0., 1.]])
```

```
In [136... a = np.identity(6)
```

```
In [138... print(a)
```

```
[[1. 0. 0. 0. 0. 0.]  
 [0. 1. 0. 0. 0. 0.]  
 [0. 0. 1. 0. 0. 0.]  
 [0. 0. 0. 1. 0. 0.]  
 [0. 0. 0. 0. 1. 0.]  
 [0. 0. 0. 0. 0. 1.]
```

```
In [2]: import numpy as np
```

```
In [4]: b = np.eye(4)
print('Identity matrix :\n',b)
```

```
Identity matrix :
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
```

```
In [5]: c = np.eye(3)
c
```

```
Out[5]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

Slicing in matrix

```
In [6]: a = np.random.randint(10,30,(5,4))
a
```

```
Out[6]: array([[17, 20, 11, 19],
               [20, 10, 14, 17],
               [13, 21, 13, 26],
               [27, 29, 26, 23],
               [10, 15, 24, 11]], dtype=int32)
```

```
In [7]: print(a)
```

```
[[17 20 11 19]
 [20 10 14 17]
 [13 21 13 26]
 [27 29 26 23]
 [10 15 24 11]]
```

```
In [9]: a[:]      #print all elements
```

```
Out[9]: array([[17, 20, 11, 19],
               [20, 10, 14, 17],
               [13, 21, 13, 26],
               [27, 29, 26, 23],
               [10, 15, 24, 11]], dtype=int32)
```

```
In [11]: a[1:3]    # print row 1 to row (3-1)
```

```
Out[11]: array([[20, 10, 14, 17],
               [13, 21, 13, 26]], dtype=int32)
```

```
In [12]: a[1:-1]
```

```
Out[12]: array([[20, 10, 14, 17],
               [13, 21, 13, 26],
               [27, 29, 26, 23]], dtype=int32)
```

```
In [13]: a[1:]
```

```
Out[13]: array([[20, 10, 14, 17],
               [13, 21, 13, 26],
               [27, 29, 26, 23],
               [10, 15, 24, 11]], dtype=int32)
```

```
In [14]: a[:3]
```

```
Out[14]: array([[17, 20, 11, 19],
               [20, 10, 14, 17],
               [13, 21, 13, 26]], dtype=int32)
```

```
In [15]: a[-1:]
```

```
Out[15]: array([[10, 15, 24, 11]], dtype=int32)
```

```
In [16]: a[:-1]
```

```
Out[16]: array([[17, 20, 11, 19],
               [20, 10, 14, 17],
               [13, 21, 13, 26],
               [27, 29, 26, 23]], dtype=int32)
```

```
In [17]: a[0:-2]
```

```
Out[17]: array([[17, 20, 11, 19],
               [20, 10, 14, 17],
               [13, 21, 13, 26]], dtype=int32)
```

```
In [36]: a[0:3:2]
```

```
Out[36]: array([[17, 20, 11, 19],
               [13, 21, 13, 26]], dtype=int32)
```

```
In [37]: a[0:3:3]
```

```
Out[37]: array([[17, 20, 11, 19]], dtype=int32)
```

Indexing in matrix

```
In [19]: print(a)
```

```
[[17 20 11 19]
 [20 10 14 17]
 [13 21 13 26]
 [27 29 26 23]
 [10 15 24 11]]
```

```
In [20]: a[1,3]      #first row and 3rd column value
```

```
Out[20]: np.int32(17)
```

```
In [22]: a[0,3]      # 0th row and 3rd column value
```

```
Out[22]: np.int32(19)
```

```
In [23]: a[1,-1]
```

```
Out[23]: np.int32(17)
```

```
In [24]: a[-1,1]
```

```
Out[24]: np.int32(15)
```

```
In [25]: a[-1,0]
```

```
Out[25]: np.int32(10)
```

```
In [26]: mat = np.random.randint(10,40,(6,6))  
print(mat)
```

```
[[31 38 34 23 26 31]  
 [36 31 26 13 18 36]  
 [29 20 31 25 17 32]  
 [23 16 39 24 37 34]  
 [23 23 16 15 32 21]  
 [18 26 27 36 38 16]]
```

```
In [27]: mat[2]      # return 2nd row only
```

```
Out[27]: array([29, 20, 31, 25, 17, 32], dtype=int32)
```

```
In [28]: mat[0]
```

```
Out[28]: array([31, 38, 34, 23, 26, 31], dtype=int32)
```

```
In [31]: mat[:,1]      # return 1st column only
```

```
Out[31]: array([38, 31, 20, 16, 23, 26], dtype=int32)
```

```
In [32]: mat[:, -1]
```

```
Out[32]: array([31, 36, 32, 34, 21, 16], dtype=int32)
```

```
In [33]: mat[-3]
```

```
Out[33]: array([23, 16, 39, 24, 37, 34], dtype=int32)
```

```
In [34]: mat[2:4,2:6]
```

```
Out[34]: array([[31, 25, 17, 32],  
                [39, 24, 37, 34]], dtype=int32)
```

```
In [35]: mat[1:3,2:5]
```

```
Out[35]: array([[26, 13, 18],  
                [31, 25, 17]], dtype=int32)
```

```
In [ ]:
```